ByteMart Digital

THE INTERVIEW PREP SERIES

Hibernate & JPA Interview Q&A

ORM core, mapping, associations, fetching, caching, transactions, queries & coding

97 interview questions with concise, accurate model answers, including 6 hands-on coding challenges with working solutions — organized by topic for focused preparation.

97	6	16	Model
Questions	Coding	Sections	Answers

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Hibernate & JPA Interview Questions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

Core Concepts 4

Mapping & Identifiers 7

Associations & Relationships 10

Fetching & Performance 13

Caching 16

Transactions & Concurrency 18

Queries 20

Inheritance & Advanced 22

Hibernate-Specific Features 24

Mapping Deep-Dive 26

Spring Data JPA 28

Lifecycle & Callbacks 30

Inheritance & Filtering Details 32

Performance & Production 34

Miscellaneous & Gotchas 36

Coding Challenges 38

This bank contains 97 Hibernate & JPA interview questions across 16 sections, including 6 coding challenges with full solutions. Use the bookmarks panel to jump to any section or question.

SECTION 01

Core Concepts

Q1 What is Hibernate, and how does it relate to JPA?

ANSWER

Hibernate is an object-relational mapping (ORM) framework that maps Java objects to relational database tables, generating the SQL and mapping results back to objects so you work with objects instead of raw JDBC. **JPA** (Jakarta Persistence API) is the Java specification for ORM — a set of interfaces and annotations — and Hibernate is its most widely used implementation.

Coding against JPA interfaces (`EntityManager`, JPQL, standard annotations) keeps your code portable across providers, while Hibernate supplies the engine and some powerful extensions beyond the spec.

> **KEY IDEA**
> JPA is the standard contract; Hibernate is the implementation. Program to JPA for portability, drop to Hibernate features when you need them.

Q2 What are the advantages and disadvantages of using an ORM?

ANSWER

An ORM removes vast amounts of boilerplate JDBC code, provides database-independence through dialects, adds caching and lazy loading, and lets you think in terms of objects and relationships rather than tables and joins.

- **Pros** — less boilerplate, portability, caching, dirty checking, association handling.
- **Cons** — a learning curve, hidden performance traps (N+1, lazy exceptions), and less control over the exact SQL.

> **WATCH OUT**
> An ORM does not free you from understanding SQL — the worst performance problems come from developers who treat it as a black box and never look at the generated queries.

Q3 Explain the entity lifecycle states.

ANSWER

A JPA entity exists in one of four states relative to the persistence context (the session).

State	Meaning
Transient	new object, not associated with a session
Persistent / managed	associated with a session; changes are tracked
Detached	was managed, but the session closed
Removed	marked for deletion on flush

A newly created object is **transient**; persisting it makes it **managed**, after which Hibernate tracks and flushes its changes automatically. When the session closes, it becomes **detached** and its changes are no longer tracked.

Q4 What is the difference between get() and load()?

ANSWER

`get()` queries the database immediately and returns the entity, or `null` if it does not exist. `load()` returns a lazy **proxy** without hitting the database; the query happens only when a property is first accessed, and if the row is missing it throws `ObjectNotFoundException` at that point.

```java
User u1 = session.get(User.class, id);    // DB hit now; null if absent
User u2 = session.load(User.class, id);   // proxy; DB hit on first access
```

TIP

Use `get()` when you are unsure the entity exists; use `load()` only to obtain a reference (e.g. to set a foreign key) without needing to read it.

Q5 What is the difference between save(), persist(), merge(), and saveOrUpdate()?

ANSWER

`persist()` is the JPA method that makes a transient entity managed, returning void. Hibernate's `save()` inserts and returns the generated identifier. `merge()` copies the state of a detached entity onto a managed instance and returns that managed copy. `saveOrUpdate()` chooses insert or update based on whether the entity is new.

WATCH OUT

After `merge()`, continue working with the **returned** instance — the object you passed in stays detached and its later changes are not tracked.

© 2026 ByteMart Digital

Q6 What is dirty checking?

ANSWER

Dirty checking is Hibernate's automatic change detection. When an entity is loaded, Hibernate keeps a snapshot of its state; at flush time (usually on commit) it compares the current state to the snapshot and generates UPDATE statements only for the fields that changed. You never call an explicit save for modifications to an already-managed entity.

```java
@Transactional
public void ship(Long id) {
    Order o = repo.findById(id).orElseThrow();
    o.setStatus("SHIPPED");   // no save() call — dirty checking flushes the update
}
```

Q7 What is the difference between Session and EntityManager?

ANSWER

`EntityManager` is the JPA-standard interface representing the persistence context, while `Session` is Hibernate's native, richer equivalent. In JPA-based applications you use `EntityManager` for portability, and can `unwrap` to a `Session` when you need a Hibernate-specific capability.

Q8 What is the SessionFactory / EntityManagerFactory, and is it thread-safe?

ANSWER

The `SessionFactory` (JPA's `EntityManagerFactory`) is a heavyweight, expensive-to-create, thread-safe object that holds configuration and mappings and produces sessions. You create **one per database** for the application's lifetime. Individual sessions, by contrast, are lightweight and **not** thread-safe — one per unit of work.

> **WATCH OUT**
> Never share a `Session`/`EntityManager` across threads — it represents a single unit of work and is not thread-safe. Only the factory is.

SECTION 02

Mapping & Identifiers

Q9 How do you map a basic entity?

ANSWER

Annotate a class with `@Entity` and give it an identifier field marked `@Id`. Optionally use `@Table` to name the table and `@Column` to customise column mapping. By convention, unmarked fields map to columns of the same name.

java

```java
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = 20)
    private String status;
}
```

Q10 What are the identifier generation strategies?

ANSWER

`@GeneratedValue` chooses how primary keys are generated.

Strategy	How it works
IDENTITY	uses an auto-increment column
SEQUENCE	uses a database sequence (preferred)
TABLE	uses a separate key table (slow)
AUTO	provider picks based on the dialect

WATCH OUT

`IDENTITY` disables JDBC batch inserts because the id is only known after each insert. For high-throughput inserts, `SEQUENCE` with a pooled optimiser is usually better.

© 2026 ByteMart Digital

Q11 What is the difference between @Embeddable and an entity?

ANSWER

An `@Embeddable` is a reusable value type with no identity of its own — its fields are stored as columns in the owning entity's table, mapped via `@Embedded`. An entity has its own identity and table. Use embeddables for value objects like an address or money amount that belong to an entity.

java

```java
@Embeddable
public record Address(String street, String city, String zip) { }

@Entity class Customer {
    @Id Long id;
    @Embedded Address address;   // columns live in the customer table
}
```

Q12 How do you map an enum, and what is the pitfall?

ANSWER

Use `@Enumerated` to map an enum field. `EnumType.STRING` stores the name; `EnumType.ORDINAL` stores the position. Always prefer `STRING`.

WATCH OUT

`ORDINAL` stores the enum's index, so reordering or inserting a constant silently corrupts existing data. `STRING` is stable against such changes.

Q13 How do you map composite primary keys?

ANSWER

A composite key uses multiple columns. Model it with an `@Embeddable` id class referenced by `@EmbeddedId`, or with a separate id class referenced by `@IdClass`. The id class must implement `equals` and `hashCode` and be serializable.

java

```java
@Embeddable
public record OrderLineId(Long orderId, Long productId) { }

@Entity class OrderLine {
    @EmbeddedId private OrderLineId id;
}
```

© 2026 ByteMart Digital

Q14 What does @Transient do?

ANSWER

`@Transient` marks a field that should **not** be persisted — it exists only in memory, for derived or temporary values. Hibernate skips it during mapping, so no column is created or read for it.

Q15 What is the difference between @Column(nullable=false) and a Bean Validation @NotNull?

ANSWER

`@Column(nullable = false)` affects the generated DDL and is a database-level constraint. A Bean Validation `@NotNull` is checked in the application before the entity is persisted, producing a friendlier validation error. Using both gives defence in depth — early validation plus a database guarantee.

Q16 What is the purpose of @Version in mapping?

ANSWER

A `@Version` field (an int, long, or timestamp) enables optimistic locking. Hibernate reads it, and on update includes it in the WHERE clause and increments it; if another transaction changed the row first, zero rows match and an `OptimisticLockException` is thrown, preventing a lost update.

© 2026 ByteMart Digital

SECTION 03

Associations & Relationships

Q17 What are the four association mappings?

ANSWER

Hibernate maps relationships between entities with four annotations, reflecting the cardinality of the relationship.

Annotation	Relationship
@OneToOne	one entity to exactly one other
@OneToMany	one entity to many
@ManyToOne	many entities to one
@ManyToMany	many to many (via a join table)

Q18 What is the difference between the owning side and the inverse side?

ANSWER

In a bidirectional association, the **owning side** holds the foreign key and controls how the relationship is persisted; the **inverse side** merely mirrors it and is marked with `mappedBy`. Hibernate only looks at the owning side when saving — changes made only to the inverse side are ignored.

java

```
@Entity class Order {
    @OneToMany(mappedBy = "order")     // inverse side
    private List<Item> items = new ArrayList<>();
}
@Entity class Item {
    @ManyToOne private Order order;    // owning side — holds the FK
}
```

WATCH OUT

Update the owning side (or a helper method that sets both) when changing a bidirectional relationship, or your change will not be persisted.

Q19 What is the difference between unidirectional and bidirectional associations?

ANSWER

A **unidirectional** association is navigable from one side only; a **bidirectional** one is navigable from both, with a `mappedBy` on the inverse side. Bidirectional gives convenient navigation but requires you to keep both sides in sync, typically via helper methods.

```java
public void addItem(Item item) {   // sync both sides
    items.add(item);
    item.setOrder(this);
}
```

Q20 How do you map a many-to-many relationship?

ANSWER

Use `@ManyToMany` with a `@JoinTable` defining the link table and its two foreign-key columns. One side owns the relationship; the other uses `mappedBy`.

```java
@ManyToMany
@JoinTable(name = "student_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id"))
private Set<Course> courses = new HashSet<>();
```

TIP

When the join needs its own attributes (e.g. an enrolment date), model it as an explicit entity with two `@ManyToOne`s instead of a plain `@ManyToMany`.

Q21 What are cascade types, and which is dangerous?

ANSWER

Cascading propagates an operation from a parent entity to its associated entities — `PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, and the catch-all `ALL`. It lets you save or delete an object graph in one operation.

WATCH OUT

`CascadeType.REMOVE` (and `ALL`) deletes children when the parent is deleted — easy to apply too broadly and delete more than intended. Apply cascades deliberately, per relationship.

© 2026 ByteMart Digital

Q22 What is orphanRemoval, and how does it differ from CascadeType.REMOVE?

ANSWER

`orphanRemoval = true` deletes a child when it is **removed from the parent's collection** — it tracks the relationship itself. `CascadeType.REMOVE` only deletes children when the **parent** is deleted. Use `orphanRemoval` for true parent-owned children that cannot exist independently.

```java
@OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
           orphanRemoval = true)
private List<Item> items;
// items.remove(x) will DELETE that item row on flush
```

Q23 Why should you avoid CascadeType and FetchType.EAGER on @ManyToMany?

ANSWER

Cascading REMOVE across a many-to-many can delete shared data other entities still reference, and eager fetching pulls entire related collections on every load, causing large joins and performance problems. Keep many-to-many associations lazy and avoid remove-cascades on them.

© 2026 ByteMart Digital

SECTION 04

Fetching & Performance

Q24 What is the difference between lazy and eager fetching?

ANSWER

Lazy fetching loads an association only when it is first accessed, issuing a separate query at that point — the default for collections (`@OneToMany`, `@ManyToMany`). **Eager** fetching loads the association immediately with the parent — the default for `@ManyToOne` and `@OneToOne`.

Lazy is generally preferred because it avoids loading data you may not use, but it requires an open session at access time. Eager can waste resources and cause huge joins if applied broadly.

	Lazy	Eager
Loads	on first access	with the parent
Default for	collections	to-one associations
Risk	LazyInitializationException	over-fetching / big joins

KEY IDEA

Default to lazy and fetch what you need explicitly per query; blanket eager fetching is a common cause of performance problems.

Q25 What causes LazyInitializationException, and how do you fix it?

ANSWER

It occurs when you access a lazy association **after** the session (persistence context) has closed — typically when serializing an entity in the web layer after the transaction ended. Hibernate has no session to load the data, so it throws.

- Fetch the association within the transaction using a `JOIN FETCH` query or `@EntityGraph`.
- Map the entity to a **DTO** inside the transactional service method.
- Avoid the open-session-in-view antipattern, which hides the real problem.

WATCH OUT

Do not 'fix' it by making everything eager — that trades one problem for over-fetching. Fetch precisely what each use case needs.

Q26 What is the N+1 select problem?

ANSWER

The N+1 problem occurs when Hibernate runs one query to load N parent entities and then one additional query per parent to load a lazy association — N+1 queries total. It is the single most common Hibernate performance issue and scales terribly.

```java
// triggers N+1: 1 query for orders, then 1 per order for items
for (Order o : orderRepo.findAll())
    total += o.getItems().size();
```

Fix it by fetching the association in a single query with `JOIN FETCH`, an `@EntityGraph`, or batch fetching (`@BatchSize`), which loads associations in groups.

Q27 How does a JOIN FETCH work?

ANSWER

`JOIN FETCH` in JPQL tells Hibernate to load an association **in the same query** as the parent, using a SQL join, rather than issuing separate lazy queries. It is the primary tool for eliminating N+1 on read paths.

```java
@Query("select distinct o from Order o join fetch o.items where o.status = :s")
List<Order> findWithItems(@Param("s") String status);
```

WATCH OUT

Fetching two collections with a single query causes a Cartesian product; fetch one collection per query, and use `distinct` to de-duplicate parent rows.

Q28 What is an @EntityGraph?

ANSWER

An `@EntityGraph` declaratively specifies which associations to fetch eagerly for a particular query, without writing a fetch join. It overrides the default lazy/eager settings on a per-query basis — a clean way to avoid N+1 while keeping mappings lazy by default.

```java
@EntityGraph(attributePaths = {"items", "customer"})
List<Order> findByStatus(String status);
```

© 2026 ByteMart Digital

Q29 What is batch fetching?

ANSWER

Batch fetching reduces N+1 by loading lazy associations in **batches** rather than one at a time. With `@BatchSize(size = 20)` (or the global setting), Hibernate loads up to 20 parents' associations in a single `IN` query, cutting N queries down to N/20.

Q30 Why are DTO projections good for read performance?

ANSWER

A DTO projection selects only the columns you need directly into a lightweight object, bypassing entity loading, the persistence context, and lazy proxies. This avoids over-fetching and lazy-loading exceptions, and is significantly faster for read-only queries that feed an API.

```java
@Query("select new com.bytemart.OrderView(o.id, o.status, o.total) " +
       "from Order o where o.customerId = :c")
List<OrderView> viewsFor(@Param("c") Long customerId);
```

SECTION 05
Caching

Q31 What is the first-level cache?

ANSWER

The first-level (L1) cache is the **persistence context** itself — scoped to a single session/transaction and always on. Within one session, requesting the same entity by id twice returns the same instance from memory without a second query, and it guarantees object identity for managed entities.

Q32 What is the second-level cache?

ANSWER

The second-level (L2) cache is **optional** and shared across sessions at the `SessionFactory` level. Backed by a provider like Ehcache or Hazelcast, it caches entities and collections across transactions, reducing database hits for frequently-read, rarely-changed data.

```java
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Country { /* good L2 candidate — reference data */ }
```

WATCH OUT

The L2 cache is only worthwhile for read-mostly data; caching frequently-updated entities adds invalidation overhead and risks stale reads.

Q33 What is the query cache?

ANSWER

The query cache stores the result identifiers of a query keyed by the query and its parameters, so repeating the same query avoids re-execution. It depends on the L2 cache to hold the actual entities and must be enabled explicitly. It only helps queries run repeatedly with the same parameters.

Q34 What are the cache concurrency strategies?

ANSWER

The L2 cache offers strategies that trade consistency for performance: `READ_ONLY` (immutable data, fastest), `NONSTRICT_READ_WRITE` (occasional stale reads acceptable), `READ_WRITE` (maintains consistency with soft locks), and `TRANSACTIONAL` (full transactional cache with a JTA provider). Choose based on how the data is updated.

© 2026 ByteMart Digital

SECTION 06

Transactions & Concurrency

Q35 What are flush modes, and when does Hibernate flush?

ANSWER

Flushing synchronises the persistence context with the database by executing pending SQL. By default (`AUTO`), Hibernate flushes before a query that might be affected by pending changes and at commit. `COMMIT` mode flushes only at commit. You rarely flush manually, but understanding when it happens explains why some changes appear mid-transaction.

Q36 What is the difference between optimistic and pessimistic locking?

ANSWER

Optimistic locking assumes conflicts are rare: it uses a `@Version` column and only detects a conflict at update time, throwing if the row changed meanwhile. **Pessimistic locking** acquires a database lock when reading (`SELECT ... FOR UPDATE`), blocking others until you commit.

	Optimistic	Pessimistic
Assumes	conflicts rare	conflicts likely
Mechanism	version check	database row lock
Cost	low (no locks)	higher (contention)

> **TIP**
> Prefer optimistic locking for typical web workloads; reserve pessimistic locking for short, high-contention critical sections like inventory decrements.

Q37 How do you apply a pessimistic lock in JPA?

ANSWER

Request a lock mode when reading the entity, e.g. `PESSIMISTIC_WRITE`, which issues `SELECT ... FOR UPDATE`. Spring Data exposes this via `@Lock` on a repository method.

```java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select p from Product p where p.id = :id")
Product findForUpdate(@Param("id") Long id);
```

Q38 What are transaction isolation levels?

ANSWER

Isolation levels control how much one transaction sees of others' uncommitted or concurrent work, trading consistency against concurrency.

Level	Prevents
READ UNCOMMITTED	nothing (dirty reads possible)
READ COMMITTED	dirty reads
REPEATABLE READ	dirty + non-repeatable reads
SERIALIZABLE	all anomalies (lowest concurrency)

Most applications run at `READ COMMITTED`, relying on optimistic locking for the rest — higher levels reduce throughput.

Q39 How does @Transactional interact with the persistence context?

ANSWER

A transaction typically bounds one persistence context. Inside it, loaded entities are managed and dirty-checked, so changes flush on commit without explicit saves. When the transaction ends, entities detach. This is why loading, modifying, and reading associations should happen within the transactional boundary.

Q40 What is the difference between save and flush?

ANSWER

`save/persist` registers an entity with the persistence context but does not necessarily hit the database immediately. `flush` forces the pending SQL (inserts, updates, deletes) to execute against the database, though the transaction is still not committed until it ends. Flushing is mostly automatic.

SECTION 07
Queries

Q41 What is JPQL, and how does it differ from SQL?

ANSWER

JPQL (Jakarta Persistence Query Language) is an object-oriented query language that operates on **entities and their fields** rather than tables and columns. It is portable across databases because Hibernate translates it into the dialect-specific SQL. It supports joins, aggregation, and projections in entity terms.

```java
@Query("select o from Order o where o.customer.city = :city")
List<Order> byCity(@Param("city") String city);
```

Q42 What is the Criteria API, and when is it useful?

ANSWER

The Criteria API builds queries programmatically and type-safely using Java objects instead of strings. It shines for **dynamic queries** where the filters depend on runtime input, letting you conditionally add predicates — something awkward with static JPQL strings.

```java
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Order> q = cb.createQuery(Order.class);
Root<Order> o = q.from(Order.class);
q.where(cb.equal(o.get("status"), "PAID"));
List<Order> result = em.createQuery(q).getResultList();
```

Q43 When would you use a native SQL query?

ANSWER

Use native SQL when you need database-specific features, complex queries JPQL cannot express, or hand-tuned performance. It sacrifices portability, so reserve it for the cases that genuinely need it. In Spring Data, set `nativeQuery = true` on `@Query`.

```java
@Query(value = "select * from orders where created_at > now() - interval '7 days'",
       nativeQuery = true)
List<Order> recent();
```

Q44 What are named queries?

ANSWER

A named query is a predefined, named JPQL (or native) query declared with `@NamedQuery` on an entity. It is validated at startup (catching syntax errors early) and reusable by name, which suits frequently-used static queries.

Q45 How do you implement pagination in JPA?

ANSWER

Set the first result and maximum results on the query (`setFirstResult/setMaxResults`), or in Spring Data pass a `Pageable` to get a `Page` with content and total counts. Hibernate translates this into the database's LIMIT/OFFSET equivalent.

```java
Page<Order> page = orderRepo.findByStatus("PAID",
                      PageRequest.of(0, 20, Sort.by("createdAt").descending()));
```

WATCH OUT

Deep OFFSET pagination is slow on large tables; for very large datasets, prefer keyset ('seek') pagination on an indexed column.

Q46 What is a @Modifying query?

ANSWER

In Spring Data, a bulk update or delete written as JPQL must be annotated `@Modifying` (and run in a transaction). It executes directly against the database in one statement, which is efficient — but it bypasses the persistence context, so managed entities in memory may become stale.

```java
@Modifying
@Query("update Order o set o.status = 'CLOSED' where o.createdAt < :cutoff")
int closeOld(@Param("cutoff") Instant cutoff);
```

WATCH OUT

Because bulk updates skip dirty checking and the L1 cache, clear or refresh affected entities afterward to avoid working with stale state.

© 2026 ByteMart Digital

SECTION 08

Inheritance & Advanced

Q47 What are the entity inheritance strategies?

ANSWER

JPA offers three ways to map an inheritance hierarchy to tables, each trading normalization against query cost.

Strategy	Mapping
SINGLE_TABLE	one table, a discriminator column
JOINED	a table per class, joined by key
TABLE_PER_CLASS	one full table per concrete class

`SINGLE_TABLE` is fastest to query (no joins) but leaves nullable columns; `JOINED` is normalized but needs joins; `TABLE_PER_CLASS` avoids joins per type but complicates polymorphic queries.

Q48 What is @MappedSuperclass?

ANSWER

`@MappedSuperclass` defines common fields and mappings (like id, audit timestamps) in a base class that subclasses **inherit**, but the base class is not itself an entity and has no table. Unlike inheritance strategies, it is not polymorphic — it is purely for sharing mapping definitions.

```java
@MappedSuperclass
public abstract class BaseEntity {
    @Id @GeneratedValue Long id;
    @CreatedDate Instant createdAt;
}
```

Q49 Why is implementing equals() and hashCode() on entities tricky?

ANSWER

Using a generated id in `equals/hashCode` is unstable: a transient entity's id is null before persistence and changes after, which breaks its behaviour in a `Set` across the lifecycle. Using all fields is also problematic for lazy associations.

TIP
The common recommendation is to base equality on a stable **business key** (a natural unique attribute) if one exists, and otherwise return a constant `hashCode` and compare by id carefully — or avoid putting unpersisted entities in hash-based collections.

Q50 What is the open-session-in-view antipattern?

ANSWER
Open-Session-in-View keeps the persistence context open until the web view (or JSON serialization) renders, so lazy associations can be loaded there. It conveniently hides `LazyInitializationException` but at a cost: it holds database connections longer, encourages N+1 in the view layer, and blurs the transactional boundary.

WATCH OUT
Disable it (`spring.jpa.open-in-view=false`) and fetch what the view needs explicitly — the exceptions it surfaces reveal real fetching problems worth fixing.

Q51 What is the difference between @OneToMany with and without a join table?

ANSWER
A bidirectional `@OneToMany`/`@ManyToOne` uses a foreign key on the 'many' side — the natural, efficient mapping. A unidirectional `@OneToMany` without `mappedBy` defaults to a separate join table, which adds an extra table and can generate inefficient delete-and-reinsert SQL. Prefer the bidirectional foreign-key mapping.

© 2026 ByteMart Digital

SECTION 09

Hibernate-Specific Features

Q52 What is @NaturalId, and why use it?

ANSWER

A `@NaturalId` marks a business key that uniquely identifies an entity in the real world (an ISBN, an email, a SKU), distinct from the surrogate primary key. Hibernate provides an efficient, cacheable lookup by natural id, and it documents the true business identity of the entity.

java

```java
@Entity
public class Book {
    @Id @GeneratedValue Long id;
    @NaturalId private String isbn;   // business key
}
// Book b = session.byNaturalId(Book.class).using("isbn", isbn).load();
```

Q53 What do @DynamicUpdate and @DynamicInsert do?

ANSWER

By default Hibernate generates static SQL that includes every column. `@DynamicUpdate` makes it generate UPDATE statements containing only the columns that actually changed, and `@DynamicInsert` omits columns with null values on insert. They can help with wide tables or when column-level triggers matter, at the cost of runtime SQL generation.

Q54 What is an AttributeConverter?

ANSWER

An `AttributeConverter` transforms a field between its Java representation and its database column form — for example, storing a custom type, an encrypted string, or a comma-joined list. Annotate the field or converter with `@Convert` to apply it.

java

```java
@Converter
public class MoneyConverter implements AttributeConverter<Money, Long> {
    public Long convertToDatabaseColumn(Money m) { return m.paise(); }
    public Money convertToEntityAttribute(Long v) { return Money.ofPaise(v); }
}
```

Q55 What is a @Formula?

ANSWER

A Hibernate `@Formula` maps a read-only property to a SQL expression computed at query time rather than a stored column — useful for derived values like a computed total or age. It is evaluated by the database whenever the entity is loaded.

Q56 What is the difference between a StatelessSession and a regular Session?

ANSWER

A `StatelessSession` has no persistence context, no first-level cache, no dirty checking, and no cascade handling. It is a lightweight, near-JDBC channel ideal for bulk operations over huge datasets, where the overhead and memory of a full session would be prohibitive.

> **WATCH OUT**
> Because a stateless session does not dirty-check, you must issue explicit updates — and it will not manage associations for you.

Q57 What does @Immutable do?

ANSWER

`@Immutable` tells Hibernate an entity or collection never changes, so it skips dirty checking for it and can cache it more aggressively. It is perfect for reference data — countries, currencies, lookup tables — that is read but never updated at runtime.

SECTION 10

Mapping Deep-Dive

Q58 What is @ElementCollection, and when do you use it?

ANSWER

`@ElementCollection` maps a collection of basic values or embeddables (not entities) into a separate table owned by the parent — for example a set of tags or phone numbers. Unlike an association, the elements have no identity of their own and are fully owned by the parent entity.

```java
@ElementCollection
@CollectionTable(name = "order_tags", joinColumns = @JoinColumn(name = "order_id"))
@Column(name = "tag")
private Set<String> tags = new HashSet<>();
```

Q59 How do you map large objects with @Lob?

ANSWER

`@Lob` maps a large object — a big text field (CLOB) or binary data like a file or image (BLOB). It signals the provider to use the appropriate large-object column type. For very large data, consider storing a reference and keeping the bytes outside the database.

Q60 How do you configure a sequence generator explicitly?

ANSWER

Pair `@GeneratedValue(strategy = SEQUENCE)` with a `@SequenceGenerator` that names the database sequence and sets an allocation size. A larger allocation size lets Hibernate reserve a block of ids, reducing round-trips during bulk inserts.

```java
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "order_seq")
@SequenceGenerator(name = "order_seq", sequenceName = "order_sequence",
                   allocationSize = 50)
private Long id;
```

Q61 What are @CreationTimestamp and @UpdateTimestamp?

ANSWER

These Hibernate annotations automatically set a field to the current time on insert (`@CreationTimestamp`) and on every update (`@UpdateTimestamp`), without you writing the assignment. They are a lightweight alternative to full JPA auditing for simple timestamping.

Q62 How do you map a one-to-one relationship efficiently?

ANSWER

A `@OneToOne` typically shares a foreign key: the owning side has `@JoinColumn` and the inverse side uses `mappedBy`. Note that a lazy `@OneToOne` on the inverse (non-owning) side often cannot be proxied and loads eagerly — a subtle gotcha. Sharing the primary key (`@MapsId`) is the most efficient mapping.

Q63 What is the difference between FetchType on collections vs to-one associations?

ANSWER

Collections (`@OneToMany`, `@ManyToMany`) default to **lazy**, while to-one associations (`@ManyToOne`, `@OneToOne`) default to **eager**. Best practice is to make everything lazy explicitly and fetch what you need per query — eager to-one associations are a frequent hidden source of extra joins.

```java
@ManyToOne(fetch = FetchType.LAZY)   // override the eager default
private Customer customer;
```

SECTION 11

Spring Data JPA

Q64 What is the difference between CrudRepository, PagingAndSortingRepository, and JpaRepository?

ANSWER

They form a hierarchy of increasing capability. `CrudRepository` gives basic CRUD; `PagingAndSortingRepository` adds paging and sorting; `JpaRepository` adds JPA-specific methods like `saveAll`, `flush`, `saveAndFlush`, and batch deletes. Most applications extend `JpaRepository`.

Interface	Adds
CrudRepository	save, findById, delete
PagingAndSortingRepository	paging + sorting
JpaRepository	flush, batch, JPA extras

Q65 What is the difference between save and saveAndFlush?

ANSWER

`save` registers the change in the persistence context but leaves flushing to the transaction boundary. `saveAndFlush` forces the SQL to execute immediately, which is useful when you need the database state updated at once — for instance before a native query in the same transaction that must see the change.

Q66 How do you write a custom repository implementation?

ANSWER

Define a custom interface, implement it in a class named with the `Impl` suffix (injecting the `EntityManager`), and have your repository extend both the Spring Data interface and your custom one. Spring wires the custom implementation in — the standard way to add complex, hand-written queries alongside derived methods.

Q67 What is the difference between interface and class (DTO) projections?

ANSWER

An **interface projection** declares getters and Spring supplies a proxy backed by the selected columns — minimal code. A **class (DTO) projection** uses a constructor expression to build a concrete object. Class projections are explicit and work well with records; interface projections are the quickest to declare.

© 2026 ByteMart Digital

Q68 How do you see the SQL Hibernate generates?

ANSWER

Enable `spring.jpa.show-sql=true` for a quick view, or better, set the logger `org.hibernate.SQL=DEBUG` and `org.hibernate.orm.jdbc.bind=TRACE` to see statements with bound parameters. Watching the SQL is the single most effective way to catch N+1 problems and unexpected queries.

> **TIP**
> In tests, tools like datasource-proxy or the Hypersistence utilities can count queries and fail the test if an operation triggers more SQL than expected.

Q69 How do you enable JDBC statement batching?

ANSWER

Set `hibernate.jdbc.batch_size` (e.g. 30) so Hibernate groups inserts and updates into batched JDBC calls instead of one round-trip each — a large throughput win for bulk writes. Note that `IDENTITY` id generation disables insert batching, which is a reason to prefer `SEQUENCE`.

Q70 How do you handle a bulk update without loading entities?

ANSWER

Use a `@Modifying` JPQL update (or a native query) which runs a single UPDATE against the database rather than loading, changing, and flushing each entity. It is far more efficient for mass changes, but it bypasses the persistence context, so clear it afterward to avoid stale managed entities.

```java
@Modifying(clearAutomatically = true)
@Query("update Product p set p.active = false where p.discontinued = true")
int deactivateDiscontinued();
```

SECTION 12

Lifecycle & Callbacks

Q71 What are JPA entity lifecycle callbacks?

ANSWER

JPA lets you hook into an entity's lifecycle with callback annotations on methods: `@PrePersist` and `@PostPersist` (around insert), `@PreUpdate/@PostUpdate`, `@PreRemove/@PostRemove`, and `@PostLoad` (after loading). They let you set defaults, normalise data, or maintain derived fields automatically.

```java
@PrePersist
void onCreate() {
    if (createdAt == null) createdAt = Instant.now();
    status = "NEW";
}
```

WATCH OUT

Keep callbacks lightweight and free of database access on other entities — doing queries inside a callback can cause subtle flush-order problems.

Q72 What is the difference between EntityManager.find() and getReference()?

ANSWER

`find()` loads the entity immediately (like Hibernate's `get`), returning null if absent. `getReference()` returns a lazy proxy (like `load`) that queries only on first property access, throwing if the row is missing. Use `getReference` to set a foreign key without loading the target.

Q73 What is the difference between merge and refresh?

ANSWER

`merge` takes a detached entity's state and copies it onto a managed instance, pushing your changes toward the database. `refresh` does the opposite — it reloads a managed entity from the database, discarding your in-memory changes and overwriting them with the current persistent state.

Q74 How does the CascadeType.DETACH and REFRESH behaviour work?

ANSWER

`DETACH` propagates detaching a parent to its children (they leave the persistence context together), and `REFRESH` propagates reloading. They round out the cascade set alongside `PERSIST`, `MERGE`, and `REMOVE`, and are applied per-association as needed.

SECTION 13

Inheritance & Filtering Details

Q75 How does the SINGLE_TABLE discriminator work?

ANSWER

With `SINGLE_TABLE`, all classes in the hierarchy share one table, and a `@DiscriminatorColumn` stores a value identifying each row's concrete type. Queries are fast (no joins), but subclass-specific columns must be nullable, since rows of other types leave them empty.

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "payment_type")
public abstract class Payment { }

@Entity @DiscriminatorValue("CARD")
public class CardPayment extends Payment { }
```

Q76 How do you implement soft deletes in Hibernate?

ANSWER

Rather than deleting rows, mark them inactive. Hibernate's `@SQLDelete` can rewrite deletes as an UPDATE of a flag, and `@Where` (or the newer `@SoftDelete`) filters out inactive rows on every query — so deleted data is retained but hidden.

```java
@Entity
@SQLDelete(sql = "update product set deleted = true where id = ?")
@Where(clause = "deleted = false")
public class Product { }
```

Q77 What is a Hibernate @Filter?

ANSWER

A `@Filter` applies a dynamic, parameterised WHERE condition that you enable per session — useful for cross-cutting concerns like multi-tenancy (show only the current tenant's rows) or region scoping, without repeating the condition in every query.

Q78 What is @OrderBy versus @OrderColumn on a collection?

ANSWER

`@OrderBy` sorts a loaded collection by one of the associated entity's properties, using an ORDER BY in the query — the order is derived from data. `@OrderColumn` persists the list's positional index in a dedicated column, preserving an explicit, user-defined ordering.

SECTION 14

Performance & Production

Q79 How do you stream a very large result set without exhausting memory?

ANSWER

Loading millions of rows into a list will exhaust the heap. Instead, use a scrollable/streamed result (`Stream` return type in Spring Data, or a `ScrollableResults`) that processes rows in batches, and periodically `flush()` and `clear()` the persistence context so processed entities are garbage-collected.

> **WATCH OUT**
> A streaming query must run inside an open transaction, and you should clear the L1 cache periodically — otherwise the persistence context grows unbounded and defeats the purpose.

Q80 What is Hibernate Envers?

ANSWER

Envers provides automatic entity auditing — annotate an entity with `@Audited` and Hibernate records a full history of every change in revision tables. You can then query the state of an entity at any past revision, which is valuable for audit trails and change tracking without hand-written history logic.

Q81 How do you diagnose and fix a connection leak?

ANSWER

A connection leak occurs when connections are borrowed from the pool but never returned, eventually exhausting it and hanging the application. HikariCP's leak-detection threshold logs stack traces of long-held connections. The usual causes are transactions that never commit or close, or manual session handling that skips cleanup — fixed by relying on Spring's declarative transactions.

Q82 Why can serializing entities to JSON cause problems?

ANSWER

Serializing entities directly can trigger `LazyInitializationException` (accessing lazy associations after the session closed), infinite recursion on bidirectional relationships, and accidental exposure of internal fields. Mapping to DTOs within the transaction avoids all three and keeps the API decoupled from the schema.

© 2026 ByteMart Digital

Q83 What is the difference between statement-level and entity-level caching?

ANSWER

The persistence-context (L1) and second-level (L2) caches operate at the **entity** level, storing objects by id. Databases and JDBC drivers also cache at the **statement** level (prepared-statement caching), reusing parsed SQL. They are complementary: entity caching cuts queries, statement caching cuts parsing cost for the queries that do run.

Q84 How do you decide between a JOIN FETCH and an @EntityGraph?

ANSWER

Both eliminate N+1 by fetching associations up front. A `JOIN FETCH` lives in the query and is explicit about the SQL join; an `@EntityGraph` is declarative and reusable across derived methods without writing JPQL. Use a graph when you want to keep repository method names and add fetching orthogonally; use a fetch join for complex, one-off queries.

SECTION 15

Miscellaneous & Gotchas

Q85 What is a Hibernate dialect, and why does it matter?

ANSWER

A dialect tells Hibernate how to generate SQL for a specific database — its data types, pagination syntax, sequence support, and functions. Setting the correct dialect (or letting Boot auto-detect it) ensures the generated SQL is valid and optimal for your database, and it is what makes the same JPQL portable across engines.

Q86 How do you map a JSON column?

ANSWER

Modern Hibernate maps a field to a native JSON/JSONB column with `@JdbcTypeCode(SqlTypes.JSON)`, serializing the object to JSON on write and back on read. This lets you store semi-structured data alongside relational columns while still querying it with database JSON functions.

```java
@JdbcTypeCode(SqlTypes.JSON)
@Column(columnDefinition = "jsonb")
private Map<String, Object> metadata;
```

Q87 What is a query hint, and when is it useful?

ANSWER

A `@QueryHint` passes provider-specific instructions to a query — marking it read-only (skipping dirty-check snapshots), setting a fetch size, enabling the query cache, or adding a timeout. Read-only hints are a common, cheap performance win for pure read queries.

Q88 Why should you avoid business logic in getters of lazy associations?

ANSWER

Accessing a lazy association triggers a database query, so calling a getter inside a loop or a serialization path can silently fire many queries (the N+1 pattern) or throw if the session has closed. Keep association access deliberate and inside the transactional boundary, fetching what you need explicitly.

© 2026 ByteMart Digital

Q89 What is the difference between a read-only and a normal query for performance?

ANSWER

A read-only query tells Hibernate not to take dirty-checking snapshots of the returned entities, cutting memory and CPU for large read operations. Combined with `@Transactional(readOnly = true)`, it signals intent clearly and can route to read replicas — a strong default for query endpoints.

Q90 What is bytecode enhancement in Hibernate?

ANSWER

Bytecode enhancement instruments entity classes at build time to enable features like lazy loading of individual basic attributes, dirty tracking without snapshots, and lazy to-one associations that would otherwise load eagerly. It is an advanced optimisation for performance-critical mappings.

Q91 How do you prevent updates from overwriting concurrent changes?

ANSWER

Use optimistic locking with a `@Version` field: Hibernate includes the version in the update's WHERE clause, so a stale update matches zero rows and fails with `OptimisticLockException` rather than silently clobbering another user's change. Handle it by reloading and retrying or reporting a conflict.

© 2026 ByteMart Digital

SECTION 16

Coding Challenges

Q92 Map a bidirectional one-to-many relationship with a sync helper.

</> CODE

SOLUTION

The `@ManyToOne` side owns the foreign key; the `@OneToMany` side is inverse with `mappedBy`. A helper keeps both sides consistent in memory.

java

```
@Entity
public class Order {
    @Id @GeneratedValue private Long id;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Item> items = new ArrayList<>();

    public void addItem(Item item) {
        items.add(item);
        item.setOrder(this);
    }
}

@Entity
public class Item {
    @Id @GeneratedValue private Long id;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id")
    private Order order;
}
```

Q93 Write a JPQL query that avoids the N+1 problem with JOIN FETCH.

</> CODE

SOLUTION

Fetch the collection in the same query with `join fetch` and use `distinct` to collapse duplicated parent rows.

java

```
public interface OrderRepository extends JpaRepository<Order, Long> {
    @Query("select distinct o from Order o " +
           "join fetch o.items where o.status = :status")
    List<Order> findWithItems(@Param("status") String status);
}
```

Q94 Add optimistic locking to an entity.

</> CODE

SOLUTION

Add a `@Version` field; JPA checks and increments it on update and throws on a concurrent modification, which you translate to a 409 conflict.

```java
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    private BigDecimal balance;
    @Version private long version;   // optimistic lock
}
// On concurrent update: OptimisticLockException -> retry or return HTTP 409
```

Q95 Write a Spring Data repository with derived and custom queries.

</> CODE

SOLUTION

Combine a derived method for a simple case with a `@Query` for a join, and return a projection where only some fields are needed.

```java
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByCategoryAndActiveTrue(String category);

    @Query("select p from Product p where p.price between :lo and :hi")
    List<Product> inPriceRange(@Param("lo") BigDecimal lo,
                               @Param("hi") BigDecimal hi);
}
```

Q96 Map a composite primary key with @EmbeddedId.

</> CODE

SOLUTION

Model the key as an `@Embeddable` (a record works well) and reference it with `@EmbeddedId`. The key type must be serializable with correct equality.

```java
@Embeddable
public record EnrollmentId(Long studentId, Long courseId)
        implements Serializable { }

@Entity
public class Enrollment {
    @EmbeddedId private EnrollmentId id;
    private Instant enrolledAt;
}
```

Q97 Build a dynamic query with the Specification API.

</> CODE

SOLUTION

Compose predicates conditionally so only the supplied filters are applied — the clean way to handle optional search criteria.

```java
public class OrderSpecs {
    public static Specification<Order> hasStatus(String s) {
        return (root, q, cb) -> cb.equal(root.get("status"), s);
    }
    public static Specification<Order> totalAtLeast(BigDecimal min) {
        return (root, q, cb) -> cb.greaterThanOrEqualTo(root.get("total"), min);
    }
}
// usage:
Specification<Order> spec = Specification.where(null);
if (status != null) spec = spec.and(OrderSpecs.hasStatus(status));
if (min != null)    spec = spec.and(OrderSpecs.totalAtLeast(min));
List<Order> result = orderRepo.findAll(spec);
```